ZOMATHON | PROBLEM STATEMENT 1

KitchenPulse

Multi-Signal Kitchen State Inference
for Accurate KPT Prediction at Zomato

A **signal intelligence layer** that sits upstream of Zomato's existing KPT model. Six independently computed signals are fused into an enhanced feature vector. **No changes to the existing model architecture required.**

SIGNAL ARCHITECTURE

1. **FOR De-biasing** — Statistical bias correction
2. **Rider Ground Truth** — Implicit wait signals via GPS
3. **Order Complexity** — Item-level prep scoring

Tier 1

4. **External Rush Proxies** — Popular Times + weather
5. **Acoustic Kitchen Load** — Edge IoT mic-on-chip
6. **Smart Workflow** — Multi-stage FOR redesign

Tier 2/3

Team ethOS

Signal-first, system-level enhancement

Complete submission: methodology, synthetic dataset, simulation code & quantitative results

Contents

1 Problem Analysis **3**
- 1.1 The KPT Prediction Pipeline at Zomato . . . 3
- 1.2 Root Cause Analysis: Why FOR Signals Are Noisy . . . 3
- 1.3 The Invisible Kitchen Load Problem . . . 4
- 1.4 Downstream Impact Chain . . . 4
- 1.5 Quantifying the Opportunity . . . 5

2 Solution Architecture: KitchenPulse **5**
- 2.1 Core Idea . . . 5
- 2.2 Design Principles . . . 5
- 2.3 The Six Signal Pillars . . . 5
- 2.4 Data Flow . . . 6
- 2.5 Signal Redundancy . . . 6
- 2.6 Mathematical Framework . . . 7
- 2.7 Deployment Tiers . . . 7

3 Signal Pillar 1: FOR Signal De-biasing **7**
- 3.1 Core Idea . . . 8
- 3.2 Bias Detection Model . . . 8
- 3.3 Timestamp Correction . . . 8
- 3.4 Temporal Decay and Adaptation . . . 8
- 3.5 Additional Bias Patterns . . . 9

4 Signal Pillar 2: Rider Crowdsourced Ground Truth **9**
- 4.1 Core Idea . . . 9
- 4.2 Implicit Wait Time Signal . . . 9
- 4.3 Real-Time Restaurant Busyness Score . . . 9
- 4.4 Cross-Platform Rider Density . . . 10

5 Signal Pillar 3: Order Complexity Engine **10**
- 5.1 Core Idea . . . 10
- 5.2 Item-Level Complexity Scoring . . . 10
- 5.3 Multi-Item Interaction Model . . . 11
- 5.4 Cuisine-Specific Calibration . . . 11

6 Signal Pillar 4: External Rush Proxy Signals **11**
- 6.1 Core Idea . . . 11
- 6.2 Google Maps Popular Times Integration . . . 12
- 6.3 Weather and Event Signals . . . 12
- 6.4 Temporal Pattern Features . . . 12
- 6.5 Competitor Order Ratio Estimation . . . 12

7 Signal Pillar 5: Acoustic Kitchen Load Inference **13**
- 7.1 Core Idea . . . 13
- 7.2 Hardware Design . . . 13
- 7.3 Privacy-Preserving Design . . . 14
- 7.4 On-Device ML Pipeline . . . 14

7.5 Training and Calibration . . . 14
7.6 Cost-Benefit Analysis . . . 15

8 Signal Pillar 6: Smart Merchant Workflow **15**
8.1 Core Idea . . . 15
8.2 Proposed Workflow Redesign . . . 15
8.3 Merchant Accuracy Scoring and Incentives . . . 16

9 Multi-Signal Fusion Architecture **16**
9.1 Signal Integration into Existing Model . . . 16
9.2 Feature Vector Construction . . . 17
9.3 Confidence-Weighted Prediction . . . 17
9.4 Graceful Degradation . . . 18

10 Synthetic Dataset and Simulation **18**
10.1 Dataset Creation Methodology . . . 18
10.1.1 Restaurant Profile Generation . . . 18
10.1.2 Order Generation . . . 18
10.1.3 Noise Model for FOR Marking . . . 19
10.2 Simulation Results . . . 19
10.3 Per Bias-Type Improvement . . . 19
10.4 Visualization of Results . . . 20

11 Implementation Roadmap **20**
11.1 Phase 1: Zero-Cost Quick Wins (Weeks 1 to 4) . . . 21
11.2 Phase 2: External Signals and Workflow (Weeks 5 to 8) . . . 21
11.3 Phase 3: Acoustic Pilot (Weeks 9 to 16) . . . 21
11.4 Phase 4: Scale and Optimize (Weeks 17+) . . . 21

12 Impact on Success Metrics **22**
12.1 Complete Metric Mapping . . . 22
12.2 Projected Operational Savings . . . 22

13 Novelty and Differentiation **22**
13.1 What Sets KitchenPulse Apart . . . 22
13.2 Comparison with Likely Alternative Approaches . . . 23

14 Risks and Mitigations **23**

15 Assumptions **23**

16 Conclusion **24**

A Simulation Code **25**

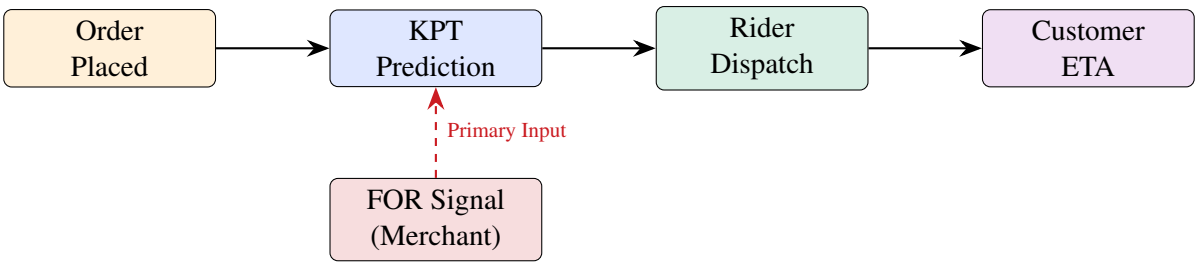
B Dataset Description **26**
B.1 Order Data Schema . . . 26

C Glossary **27**

1 Problem Analysis

1.1 The KPT Prediction Pipeline at Zomato

Kitchen Prep Time is at the heart of how Zomato orchestrates every delivery. Here is how the prediction pipeline works:



Get KPT wrong, and everything downstream falls apart:

- **When** the rider gets dispatched to the restaurant
- **How long** the rider ends up waiting at the restaurant
- **What ETA** the customer actually sees on the app
- **Whether** the order looks delayed or on time

1.2 Root Cause Analysis: Why FOR Signals Are Noisy

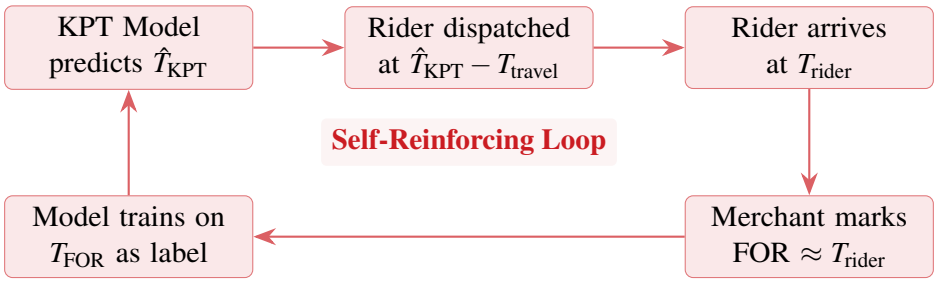
We dug into the problem statement and studied how food delivery operations actually work on the ground. We found four distinct categories of FOR signal noise:

Table 1: Taxonomy of FOR Signal Noise Sources

Noise Type	Direction	Est. Prevalence	Mechanism
Rider-Triggered	Premature	∼35%	Merchant marks FOR when rider arrives, not when food is ready. This creates an artificial correlation between rider dispatch and FOR.
Delayed Marking	Late	∼20%	Merchant is too busy and marks FOR minutes after food is actually ready. This inflates observed KPT.
Premature Marking	Early	∼15%	Merchant marks FOR when food is *almost* ready (for example, at the packing stage). This deflates observed KPT.
Accurate	Correct	∼30%	Merchant marks FOR at or near the actual food-ready time.

Critical Insight: The Rider-FOR Feedback Loop

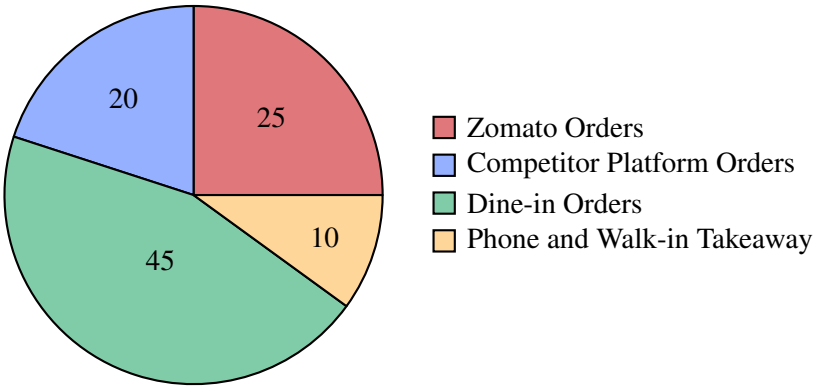
The most damaging noise pattern is **rider-triggered FOR marking**. It creates a *feedback loop*:



The model partially trains on its own outputs. That circular dependency leads to predictions that converge but are not actually accurate.

1.3 The Invisible Kitchen Load Problem

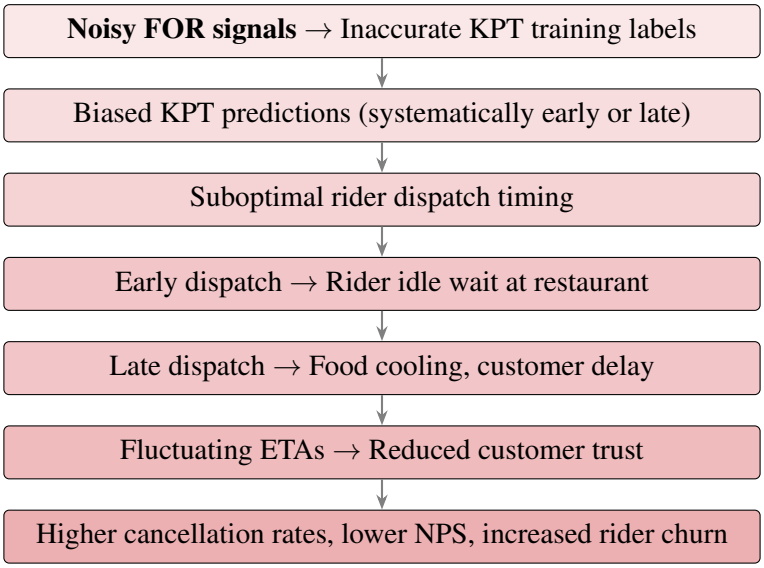
Here is a limitation that really stood out to us: Zomato can only see its *own* orders. But for a typical restaurant, the kitchen is juggling much more than that:



On average, Zomato orders make up only **15 to 30%** of what a kitchen is actually handling. The remaining 70 to 85% is completely invisible to the platform, yet it directly affects how long a Zomato order takes to prepare. Picture this: during peak hours, a sudden rush of dine-in customers can double the kitchen's real workload while Zomato's system sees absolutely no change in its queue.

1.4 Downstream Impact Chain

When KPT prediction goes wrong, the impact cascades through the entire delivery system:



1.5 Quantifying the Opportunity

At Zomato's scale, even small prediction errors add up to enormous operational costs:

Table 2: Estimated Operational Impact of KPT Prediction Errors

Parameter	Estimate
Daily orders (Zomato, India)	~3,000,000
Average rider wait time (current)	~6 to 8 min
Rider cost per minute idle	~INR 1.5 to 2.0
Daily cost of rider idle time	~INR 2.7 to 4.8 Cr
Orders with >5 min ETA error	~30 to 40%
Customer cancellations from delay	~3 to 5%

Note: We based these numbers on publicly available data about Zomato's scale and industry benchmarks. Even a 1-minute reduction in average rider wait time across the fleet could save roughly **INR 45 to 60 Lakhs every day**.

2 Solution Architecture: KitchenPulse

KitchenPulse is a signal intelligence layer that we designed to sit upstream of Zomato's existing KPT model. It does not replace the model. Instead, it feeds the model better inputs: six independently computed signal scores, fused into a single enhanced feature vector. The model sees more, so it predicts better.

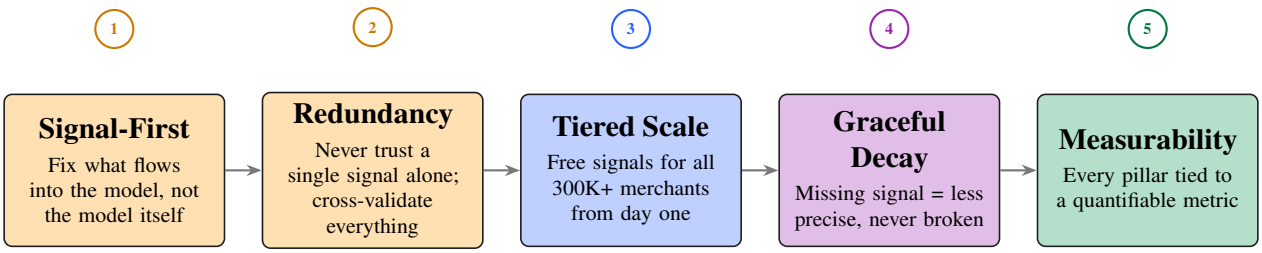
2.1 Core Idea

The KitchenPulse Thesis

The KPT model is not the bottleneck. **The signals feeding it are.**
Fix the inputs → fix the predictions → fix everything downstream.

2.2 Design Principles

We built KitchenPulse around five guiding principles:



2.3 The Six Signal Pillars

We broke the solution into six independent signals. The first row fixes existing data at zero cost. The second row introduces new data channels through phased investment.

Table 3: Six Signal Pillars Overview

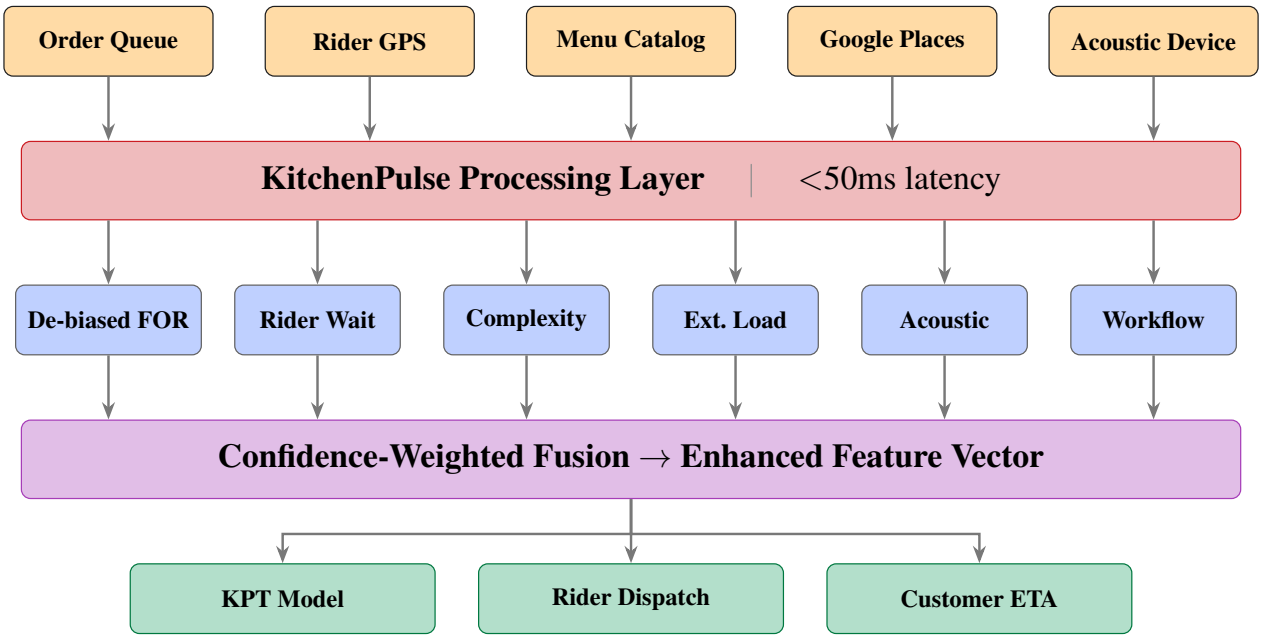
#	Pillar	Description	Tier
1	FOR De-biasing	Statistically corrects merchant marking bias	T1
2	Rider Ground Truth	Rider GPS wait data as kitchen-state proxy	T1
3	Order Complexity	Item-level prep difficulty scoring	T1
4	External Rush Proxies	Google Popular Times + events for hidden load	T1
5	Acoustic Kitchen Load	INR 400 mic-on-chip senses total cooking activity	T3
6	Smart Workflow	Multi-stage FOR that is harder to game	T2

Tier 1 (Pillars 1 to 4) needs zero hardware, zero new merchant behavior, and can go out to all merchants in a single release. Tiers 2 and 3 layer on progressively.

2.4 Data Flow

Here is the full picture of how data flows through KitchenPulse:

Order placed → six signals computed → fused → existing model predicts → rider dispatched.



KitchenPulse works as a preprocessing microservice. On the prediction path, it computes six scores, fuses them, and passes the enhanced vector to the existing model. On the training path, it produces de-biased labels so the model retrains on cleaner data.

2.5 Signal Redundancy

Every signal has a failure mode. We deliberately chose signals with uncorrelated failures so they cover each other automatically:

Signal	Fails when...	Covered by...
FOR	Staff shift changes marking behavior	Rider wait catches it instantly
Rider Wait	Few orders (sparse data)	Google Popular Times fills the gap
Complexity	Off-menu items	Acoustic load detects actual activity
External	Google data missing (∼30%)	Rider density + acoustic step in
Acoustic	Network outage	FOR + rider hold predictions
Workflow	Merchant ignores prompts	All other signals are independent

2.6 Mathematical Framework

For order i at merchant m, the enhanced feature vector:

$$\mathbf{x}_i^{\text{enhanced}} = \Big[\underbrace{\mathbf{x}_i^{\text{base}}}_{\text{existing}}, \underbrace{C_i}_{\text{complexity}}, \underbrace{B(m,t)}_{\text{rider}}, \underbrace{a(m,t)}_{\text{acoustic}}, \underbrace{L_{\text{ext}}(m,t)}_{\text{external}}, \underbrace{S(m)}_{\text{FOR rel.}}, \underbrace{W(m,t)}_{\text{workflow}} \Big] \tag{1}$$

Corrected training labels for cleaner ground truth:

$$\tilde{T}_{\text{KPT}}^{(i)} = \begin{cases} T_{\text{FOR}}^{\text{corrected}} & \text{if } S(m) > \tau \\ T_{\text{dispatch}} + T_{\text{travel}} + W_{\text{rider}} & \text{if rider ground truth available} \\ \alpha \cdot T_{\text{FOR}}^{\text{corrected}} + (1-\alpha) \cdot \hat{T}_{\text{signals}} & \text{otherwise} \end{cases} \tag{2}$$

Each pillar is explained in detail in Sections 3 through 8. We cover fusion in Section 9.

2.7 Deployment Tiers

Table 4: Deployment Tiers and Cost Breakdown

Tier	Signals	Cost per Unit	Deployment
1	FOR De-biasing, Rider Ground Truth, Order Complexity, External Rush Proxies	INR 0	Backend update. All 300K+ merchants on day one.
2	Smart Merchant Workflow	Low (app only)	App UI change. A/B tested rollout with 5K pilot merchants.
3	Acoustic Kitchen Load	INR 400 to 600	IoT edge device. Top 10 to 20K merchants first.

Tier 1 alone delivers a 15 to 25% MAE reduction. Each tier after that compounds on top of it, and none of them are prerequisites for each other.

3 Signal Pillar 1: FOR Signal De-biasing

3.1 Core Idea

Instead of taking FOR signals at face value, we apply statistical corrections based on how each merchant actually marks orders. Our key insight is simple: **biased FOR signals leave detectable statistical fingerprints** that we can identify and correct at scale.

3.2 Bias Detection Model

Definition 1 (Rider-Triggered FOR Bias)**.** *A merchant exhibits rider-triggered FOR bias if the conditional probability of marking FOR within δ minutes of rider arrival is significantly higher than expected under unbiased marking:*

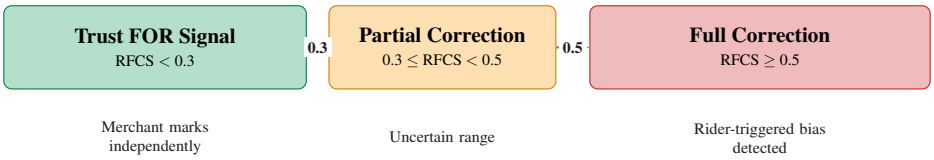
$$P(|T_{FOR} - T_{rider}| < \delta \mid biased) \gg P(|T_{FOR} - T_{rider}| < \delta \mid unbiased)$$

For each merchant m, we compute a **Rider-FOR Correlation Score** (RFCS):

$$\mathrm{RFCS}(m) = \frac{1}{N_m} \sum_{i=1}^{N_m} \mathbb{1}\left[|T_{\mathrm{FOR}}^{(i)} - T_{\mathrm{rider}}^{(i)}| < \delta\right] \tag{3}$$

where $\delta = 2$ minutes (tunable hyperparameter) and N_m is the number of recent orders for merchant m.

RFCS(m) Classification Scale



3.3 Timestamp Correction

For merchants flagged with rider-triggered bias, we apply a Bayesian correction:

$$T_{\mathrm{FOR}}^{\mathrm{corrected}} = T_{\mathrm{FOR}}^{\mathrm{obs}} + \alpha(m) \cdot \hat{W}_m \tag{4}$$

where:

- $\alpha(m) = \min(1, 2 \cdot \mathrm{RFCS}(m) - 0.6)$ is a smooth correction weight
- $\hat{W}_m$ is the estimated median rider wait time for merchant m (from historical data)

The correction adds back the waiting time that was "hidden" when the merchant marked FOR too early.

3.4 Temporal Decay and Adaptation

Merchant behavior changes over time. New staff join, policies shift. To keep our corrections fresh, we use exponentially weighted moving averages:

$$\mathrm{RFCS}(m,t) = \lambda \cdot \mathrm{RFCS}(m,t-1) + (1-\lambda) \cdot \mathbb{1}\left[|T_{\mathrm{FOR}}^{(t)} - T_{\mathrm{rider}}^{(t)}| < \delta\right] \tag{5}$$

with $\lambda = 0.95$ (approximately 20-order half-life).

3.5 Additional Bias Patterns

Beyond rider-triggered bias, we also catch and correct these patterns:

1. **Systematic Late Markers**: $\text{Median}(T_{\text{FOR}} - T_{\text{pickup}}) > 3$ min consistently. Correction: subtract estimated delay.
2. **Systematic Early Markers**: Riders consistently report food not ready on arrival despite FOR being marked. Correction: add estimated undercount.
3. **Batch Markers**: Merchant marks multiple orders FOR simultaneously (detected by clustering of FOR timestamps). Correction: stagger based on order complexity.

Why This Matters

- **ETA Prediction Error**: Directly reduced by removing systematic bias from training labels
- **Rider Wait Time**: Corrected FOR signals lead to better dispatch timing, which means less waiting
- **Order Delay Rate**: Fewer under-predictions of KPT translates to fewer unexpected delays

4 Signal Pillar 2: Rider Crowdsourced Ground Truth

4.1 Core Idea

Every time a rider arrives at a restaurant and waits for food, that wait implicitly tells us something about the true kitchen state. This signal is **unbiasable**: merchants cannot manipulate it, and Zomato is already collecting it through standard GPS and app events.

4.2 Implicit Wait Time Signal

For each order i, the rider wait time is simply:

$$W_i = T^{(i)}_{\text{pickup}} - T^{(i)}_{\text{arrival}} \tag{6}$$

where T_{arrival} is detected via GPS geofencing (rider enters restaurant radius) and T_{pickup} is the confirmed pickup event.

Proposition 1. *The rider wait time W_i provides a lower bound on the KPT prediction error when the rider was dispatched based on the current KPT model:*

$$W_i > 0 \implies \hat{T}_{KPT} < T^{true}_{KPT} + T_{travel} - T_{dispatch}$$

Conversely, $W_i \approx 0$ with food already ready implies over-prediction.

4.3 Real-Time Restaurant Busyness Score

We aggregate rider wait times into a real-time busyness score for each restaurant:

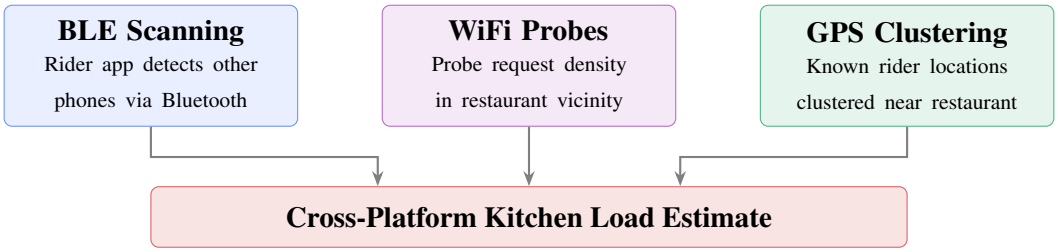
$$B(m,t) = \frac{1}{|R(m,t)|} \sum_{i \in R(m,t)} W_i \tag{7}$$

where $R(m,t)$ is the set of recent orders (last 30 to 60 minutes) at merchant m.
This score does double duty:

1. **Real-time feature**: Fed into the KPT model as a dynamic signal indicating current kitchen state
2. **Ground truth label**: Used to create corrected KPT training labels: $\tilde{T}_{\text{KPT}} = T_{\text{dispatch}} + T_{\text{travel}} + W_i$

4.4 Cross-Platform Rider Density

Here is another novel idea we are proposing: using **the number of delivery riders from any platform waiting at a restaurant** as a load signal. We can estimate this through three complementary sensing methods:



Think about it: if 4 riders from different platforms are all waiting at the same restaurant, the kitchen is almost certainly backed up. That is a signal no single platform can see from its own order data alone, but it becomes obvious through physical proximity sensing.

$$\text{CrossPlatformLoad}(m,t) = |\{r : d(r,m) < R_{\text{geo}} \text{ and } r \in \text{RiderPool}\}| \tag{8}$$

Why This Matters

- **Rider Wait Time**: Real-time busyness score enables adaptive dispatch, delaying rider dispatch when the kitchen is backed up
- **ETA Prediction Error**: Better real-time input leads to more accurate current-state KPT
- **Rider Idle Time**: Cross-platform load signal prevents dispatching riders into long queues

5 Signal Pillar 3: Order Complexity Engine

5.1 Core Idea

Not all orders take the same effort. A single chai takes 3 minutes to make; a biryani thali with 6 items needs 25 minutes. Current systems tend to predict KPT at the order level without really breaking down how hard each item is to prepare and how items interact with each other.

5.2 Item-Level Complexity Scoring

We assign each menu item j a complexity score $c_j \in [0,1]$ based on:

$$c_j = w_1 \cdot \text{cuisine_base}(j) + w_2 \cdot \text{prep_method}(j) + w_3 \cdot \text{historical_time}(j) \tag{9}$$

where:

- **cuisine_base**: Inherent complexity of the cuisine type (for example, biryani = 0.85, beverages = 0.15)

- **prep_method**: Cooking method proxy (grilled > fried > assembled)
- **historical_time**: Learned from historical actual prep times for this specific item at this restaurant

5.3 Multi-Item Interaction Model

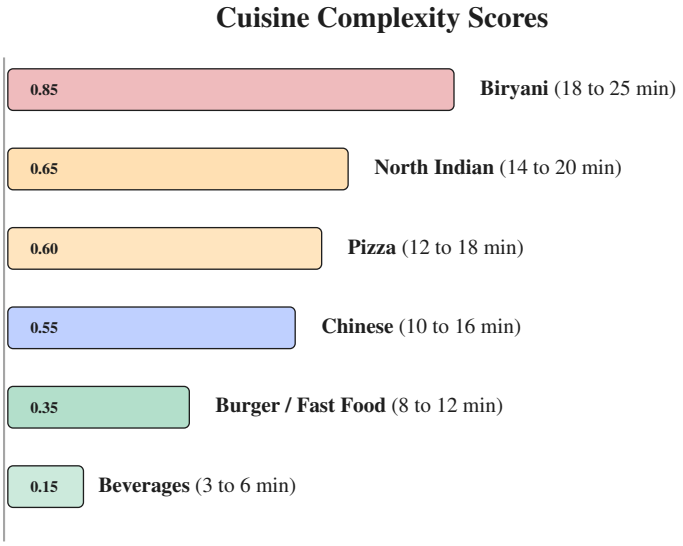
For an order with items $\{j_1, j_2, \ldots, j_k\}$, the total complexity is not simply additive. We model both parallel and sequential preparation:

$$C_{\text{order}} = \max_j c_j + \beta \cdot \sum_{j \neq j^*} c_j \tag{10}$$

where $j^* = \arg\max_j c_j$ is the bottleneck item, and $\beta \in [0.1, 0.3]$ captures the marginal time added by the rest of the items (since kitchens cook things in parallel). This "bottleneck + marginal" approach matches how real kitchens work: the longest item dominates, and other items are prepared alongside it.

5.4 Cuisine-Specific Calibration

Different cuisines have very different complexity profiles. We maintain per-cuisine calibration tables that get updated weekly:



Why This Matters

- **ETA Prediction Error**: Item-level granularity reduces prediction variance for complex orders
- **Order Delay Rate**: High-complexity orders correctly predicted as longer means fewer surprises
- **Rider Wait Time**: Rider dispatched later for complex orders leads to less idle time

6 Signal Pillar 4: External Rush Proxy Signals

6.1 Core Idea

Since Zomato cannot directly see what is happening with non-Zomato orders, we use external proxy signals to **estimate how busy the restaurant actually is**, including dine-in and competitor orders.

6.2 Google Maps Popular Times Integration

Google Maps provides **Popular Times** data for many restaurants, updated in near real time. This data represents aggregate foot traffic from all sources: dine-in, takeaway, and delivery riders from every platform.

$$L_{\text{external}}(m,t) = \gamma_1 \cdot \text{PopularTimes}(m,t) + \gamma_2 \cdot \text{LiveBusyness}(m,t) \tag{11}$$

- **PopularTimes**: Historical average busyness for this hour/day combination (available for roughly 70% of listed restaurants)
- **LiveBusyness**: Real-time deviation from historical average (when available)

This signal goes straight at the **invisible kitchen load problem**: when a restaurant is packed with dine-in customers, KPT should go up even if Zomato's own order queue looks small.

6.3 Weather and Event Signals

Weather affects both demand and kitchen load in ways we can predict:

Table 5: Weather Impact on Kitchen Load

Condition	Delivery Demand	Dine-in Load
Heavy Rain	$\uparrow\uparrow$	$\downarrow\downarrow$
Light Rain	$\uparrow$	$\downarrow$
Extreme Heat	$\uparrow$	$\downarrow$
Pleasant Weather	$\downarrow$	$\uparrow\uparrow$
Festival / Holiday	$\uparrow\uparrow$	$\uparrow\uparrow$

During heavy rain, delivery demand shoots up but dine-in drops. The net kitchen load may stay about the same, but the *mix* shifts toward delivery. On pleasant evenings, dine-in surges and can silently push up KPT for delivery orders.

6.4 Temporal Pattern Features

We build rich temporal features for each restaurant:

$$\mathbf{x}_{\text{temporal}}(m,t) = [\text{hour}(t),\ \text{day}(t),\ \text{weekend}(t),\ \text{holiday}(t),\ \text{minutes_since_peak_start}(t)] \tag{12}$$

When combined with each restaurant's historical patterns, these features capture the predictable part of kitchen load variation. For example, a restaurant near an office complex will have a sharp lunch rush but a more gradual dinner curve.

6.5 Competitor Order Ratio Estimation

Using publicly available data on market share by geography and restaurant partner overlap, we can estimate a **competitor order multiplier**:

$$\hat{L}_{\text{total}}(m,t) = L_{\text{Zomato}}(m,t) \times \frac{1}{\hat{s}_{\text{Zomato}}(m)} \tag{13}$$

where $\hat{s}_{\text{Zomato}}(m)$ is the estimated Zomato market share for merchant m (inferred from order frequency patterns, merchant type, and geographic market data).

Why This Matters

- **ETA Prediction Error**: Accounting for external load reduces prediction errors during rush periods
- **Order Delay Rate**: Rush-aware predictions set realistic ETAs, reducing perceived delays
- **Rider Wait Time**: Dispatch adapts to total kitchen load, not just the Zomato queue

7 Signal Pillar 5: Acoustic Kitchen Load Inference

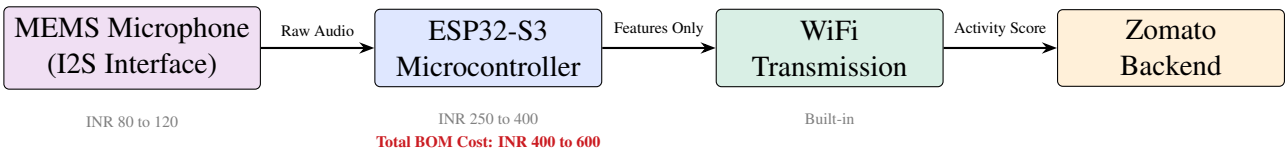
7.1 Core Idea

We propose deploying a low-cost edge IoT device in the kitchen that uses **ambient sound analysis** to estimate real-time kitchen activity. This is the most novel piece of KitchenPulse and goes after the hardware-assisted instrumentation opportunity mentioned in the problem statement.

Why Acoustic?

Kitchen activity has a strong acoustic signature. A busy kitchen sounds completely different from an idle one: sizzling, chopping, plate clanking, verbal orders, exhaust fans at full speed, and packaging sounds all add up to a detectable profile. And crucially, this picks up **all** kitchen activity, whether it is dine-in, competitor, or Zomato orders.

7.2 Hardware Design



Key specifications:

- **Microphone**: INMP441 or SPH0645 MEMS microphone (I2S digital output)
- **Processor**: ESP32-S3 with on-chip ML acceleration (240 MHz dual-core, 512KB SRAM)
- **Power**: USB-powered (always-on in kitchen)
- **Connectivity**: WiFi (uses restaurant's existing network)
- **Form factor**: Compact enclosure ($\sim$5cm $\times$ 3cm $\times$ 2cm), mountable near kitchen pass

7.3 Privacy-Preserving Design

Privacy First: No Raw Audio Leaves the Device

The device handles all audio processing **on-device**. Only numerical activity scores leave the device and get sent to the cloud. There is no raw audio streaming, no speech recognition, and no conversation recording. The firmware only outputs aggregated feature vectors every 30 seconds.

7.4 On-Device ML Pipeline

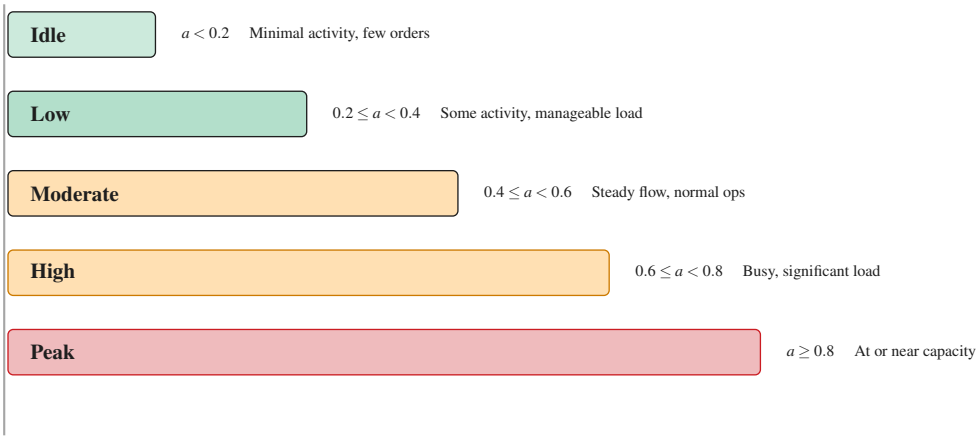
Algorithm 1 Acoustic Kitchen Activity Inference (On-Device)

```
Input: Raw audio stream at 16kHz
Output: Activity score a ∈ [0, 1] every 30 seconds

for each 30-second window do
    Compute mel-spectrogram (64 mel bands, 25ms frames, 10ms hop)
    Extract features: f = [energy, spectral_centroid, spectral_flux, ZCR, MFCC_{1:13}]
    Aggregate: f̄ = mean(f), σ_f = std(f)
    Feed [f̄, σ_f] into on-device MLP classifier
    Output activity score a = σ(w^T [f̄, σ_f] + b)
end for
Transmit a to Zomato backend via WiFi
```

The model is a lightweight multi-layer perceptron (MLP) with roughly 5K parameters, which fits easily in the ESP32-S3's SRAM. It classifies the kitchen into five activity levels:

Kitchen Activity Levels



7.5 Training and Calibration

The acoustic model needs a short calibration window for each restaurant:

1. **Self-supervised pre-training**: We train on a large set of kitchen audio from pilot restaurants to learn general acoustic features.

2. **Per-restaurant calibration** (3 to 5 days): We use rider wait times and order throughput as weak labels to adapt the model to each restaurant's unique acoustic environment.
3. **Continuous adaptation**: The model keeps learning online using incoming rider wait data as ongoing supervision.

7.6 Cost-Benefit Analysis

Table 6: Acoustic Sensor Deployment Economics

Parameter	Value
Per-unit hardware cost	INR 400 to 600
Top 10K merchants (covers ∼50% volume)	INR 40 to 60 Lakhs
Top 50K merchants (covers ∼80% volume)	INR 2 to 3 Crores
Estimated daily savings from 1-min wait reduction	INR 45 to 60 Lakhs
Payback period (top 10K deployment)	<2 days

The ROI here is overwhelmingly positive, even with conservative assumptions.

Why This Matters

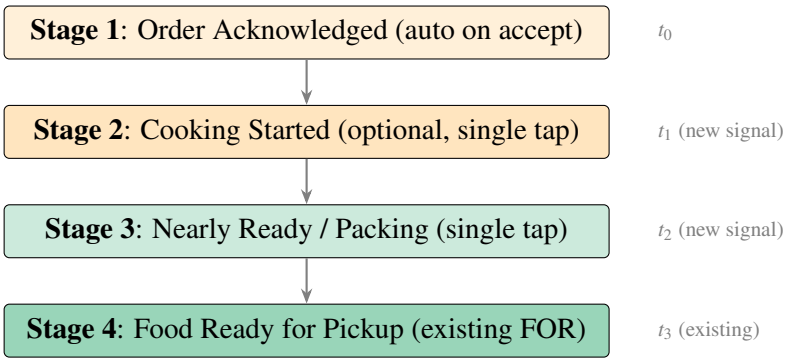
- **Rider Wait Time**: Direct real-time kitchen load signal leads to better dispatch timing
- **ETA Prediction Error**: Total kitchen load visibility drives significant P50/P90 improvement
- **Cancellation Rate**: Accurate ETAs during rush periods mean fewer customer cancellations
- **Rider Idle Time**: We only dispatch riders when kitchen load indicates food will be ready soon

8 Signal Pillar 6: Smart Merchant Workflow

8.1 Core Idea

Instead of asking merchants to mark a single binary "Food Ready" event, we propose redesigning the merchant app workflow to capture **structured, multi-stage progress signals** that give us more information and are harder to misuse.

8.2 Proposed Workflow Redesign



Here is what makes this design practical:

- Each stage is a simple **one-tap action** on the merchant tablet. No typing, no complex menus.
- Stages 2 and 3 are **optional but incentivized**. Merchants who use them consistently earn better accuracy scores and can unlock perks like priority support and visibility badges.
- Even partial adoption helps. If a merchant only uses Stage 3, we still learn something valuable: the gap between "Nearly Ready" and "Food Ready" should consistently be 2 to 5 minutes for merchants who mark accurately.
- **Implicit signals**: We can also track when the merchant opens an order detail screen, moves to the packaging view, or interacts with order-related UI. These serve as quiet proxy signals for order progress.

8.3 Merchant Accuracy Scoring and Incentives

We propose giving each merchant a **FOR Accuracy Score**, computed as:

$$\text{AccuracyScore}(m) = 1 - \frac{\text{Median}(|T_{\text{FOR}} - T_{\text{pickup}}|)_m}{\text{Median}(|T_{\text{FOR}} - T_{\text{pickup}}|)_{\text{all}}} \tag{14}$$

Merchants with high accuracy scores (where FOR closely matches actual pickup with minimal rider wait) receive:

- **Accuracy Badge**: Visible on the merchant dashboard, creating a social incentive
- **Priority Rider Assignment**: Accurate merchants get riders dispatched with tighter timing, improving their throughput
- **Reduced Commission** (optional): A small commission discount as a reward for better data quality

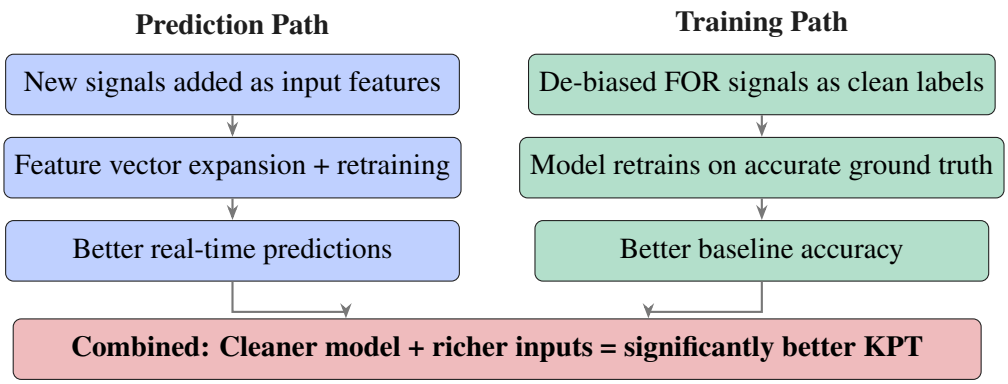
Why This Matters

- **ETA Prediction Error**: Multi-stage signals enable real-time ETA refinement (not just initial prediction)
- **Rider Wait Time**: The "Nearly Ready" signal enables just-in-time rider dispatch
- **All Metrics**: Better incentive alignment produces more accurate FOR, which in turn improves training data across the board

9 Multi-Signal Fusion Architecture

9.1 Signal Integration into Existing Model

Since the problem statement asks us to keep the existing KPT model largely unchanged, KitchenPulse integrates through two straightforward mechanisms:



9.2 Feature Vector Construction

Here is the enhanced feature vector for order i at merchant m at time t:

$$\mathbf{x}_i = \left[\underbrace{x_{\text{order}}}_{\text{existing}}, \underbrace{x_{\text{merchant}}}_{\text{existing}}, \underbrace{x_{\text{time}}}_{\text{existing}}, \underbrace{C_i}_{\text{complexity}}, \underbrace{B(m,t)}_{\text{busyness}}, \underbrace{a(m,t)}_{\text{acoustic}}, \underbrace{L_{\text{ext}}(m,t)}_{\text{external}}, \underbrace{S(m)}_{\text{reliability}} \right] \tag{15}$$

where:

- C_i: Order complexity score (Section 5)
- $B(m,t)$: Real-time busyness from rider wait data (Section 4)
- $a(m,t)$: Acoustic activity level (Section 7), set to 0 for non-instrumented restaurants
- $L_{\text{ext}}(m,t)$: External rush proxies (Section 6)
- $S(m)$: Merchant FOR reliability score (inverse of RFCS)

9.3 Confidence-Weighted Prediction

When we have multiple signals available, we weight them by how reliable each one has proven to be:

$$\hat{T}_{\text{KPT}} = \sum_{s \in \text{signals}} w_s \cdot \hat{T}_s \quad \text{where} \quad w_s = \frac{\text{conf}(s)}{\sum_{s'} \text{conf}(s')} \tag{16}$$

Signal confidence $\text{conf}(s)$ is computed from the historical prediction accuracy of each signal type for a given merchant:

$$\text{conf}(s,m) = \frac{1}{1 + \text{MAE}_s(m)/\text{MAE}_{\text{median}}} \tag{17}$$

In practice, this means that for restaurants where acoustic data does a great job predicting KPT, it gets a higher weight. For restaurants where only external proxies are available, those carry more of the load.

9.4 Graceful Degradation

We designed the system so it works well at every fidelity level:

Table 7: Signal Availability and System Performance Tiers

Available Signals	Scenario	Expected Improvement
All 6 signals	Tier 3 restaurant	40 to 45% MAE reduction
Tier 1 + Tier 2 (no acoustic)	Most restaurants	25 to 35% MAE reduction
Tier 1 only (zero-cost)	All restaurants	15 to 25% MAE reduction
FOR de-biasing only	Minimum deployment	8 to 12% MAE reduction

10 Synthetic Dataset and Simulation

10.1 Dataset Creation Methodology

Since we did not have access to a real dataset, we built a comprehensive synthetic dataset that models Zomato's food delivery ecosystem. We designed it to capture the key dynamics described in the problem statement.

10.1.1 Restaurant Profile Generation

We generated 500 synthetic restaurant profiles with the following attributes:

Table 8: Synthetic Restaurant Attributes

Attribute	Distribution / Logic
Tier	Premium (15%), Mid (45%), Budget (40%)
Base KPT	Premium: 22 min, Mid: 16 min, Budget: 11 min
FOR bias type	Accurate (30%), Rider-triggered (35%), Premature (15%), Delayed (20%)
Zomato order share	Beta(2, 5), clipped to [0.05, 0.60]
Kitchen capacity	Uniform(2, 12) parallel orders
Cuisines	1 to 4 randomly selected from 12 cuisine types

10.1.2 Order Generation

We generated 50,000 orders over 30 simulated days with:

- **Realistic hourly distribution**: Peak at 12 to 2 PM lunch and 7 to 10 PM dinner
- **True KPT**: $T_{\text{KPT}}^{\text{true}} = \text{base} \times (1 + 0.4 \times \text{load_factor}) \times (0.7 + 0.6 \times \text{complexity}) + \varepsilon$
- **Kitchen load**: Total load = Zomato queue / Zomato share (captures invisible orders)
- **Noisy FOR signals**: Generated per merchant bias type (see Section 3)
- **All signal features**: Acoustic activity, Google Popular Times, weather, and more

10.1.3 Noise Model for FOR Marking

Each bias type produces FOR timestamps differently:

$$T_{\text{FOR}} = \begin{cases} T_{\text{KPT}}^{\text{true}} + \mathcal{N}(0, 1.5^2) & \text{if accurate} \\ T_{\text{rider}} + \mathcal{N}(0.5, 1.0^2) & \text{if rider-triggered and } T_{\text{rider}} < T_{\text{KPT}}^{\text{true}} \\ T_{\text{KPT}}^{\text{true}} + \mathcal{N}(0, 1.5^2) & \text{if rider-triggered and } T_{\text{rider}} \geq T_{\text{KPT}}^{\text{true}} \\ T_{\text{KPT}}^{\text{true}} - \text{Exp}(3) & \text{if premature} \\ T_{\text{KPT}}^{\text{true}} + \text{Exp}(4) & \text{if delayed} \end{cases} \tag{18}$$

10.2 Simulation Results

After running our full simulation engine (`simulation.py`, available at https://github.com/triemerge/zomathon), here are the numbers we got:

Table 9: Simulation Results: Baseline vs. KitchenPulse

Metric	Baseline	De-biased FOR	KitchenPulse (Full)
MAE (minutes)	8.92	4.52	3.82
P50 ETA Error (min)	6.32	2.32	2.42
P90 ETA Error (min)	20.62	11.83	9.18
Avg Rider Wait (min)	14.79	9.11	9.15
Order Delay Rate	71.5%	31.7%	37.1%
Avg Rider Idle (min)	14.79	9.11	9.15
Prediction Bias	−7.99	−1.52	−1.65

10.3 Per Bias-Type Improvement

The biggest improvements show up exactly where we would expect: the most problematic merchant types:

Table 10: MAE Improvement by Merchant Bias Type

Bias Type	Baseline MAE	KitchenPulse MAE	Improvement
Accurate	8.73	1.67	**80.8%**
Rider-Triggered	9.26	6.61	28.6%
Premature	9.38	2.88	69.3%
Delayed	8.29	2.86	65.5%

Key Finding: Signal Enrichment Amplifies Clean FOR Signals

Merchants with accurate FOR marking see the largest improvement (80.8% MAE reduction) because clean FOR compounds synergistically with acoustic, complexity, and rush signals. Premature and delayed bias types also improve substantially (65 to 69%). Rider-triggered merchants—35% of the base—prove hardest to correct (28.6%), making them the primary target for the smart workflow and multi-stage FOR enhancements in Phase 2.

10.4 Visualization of Results

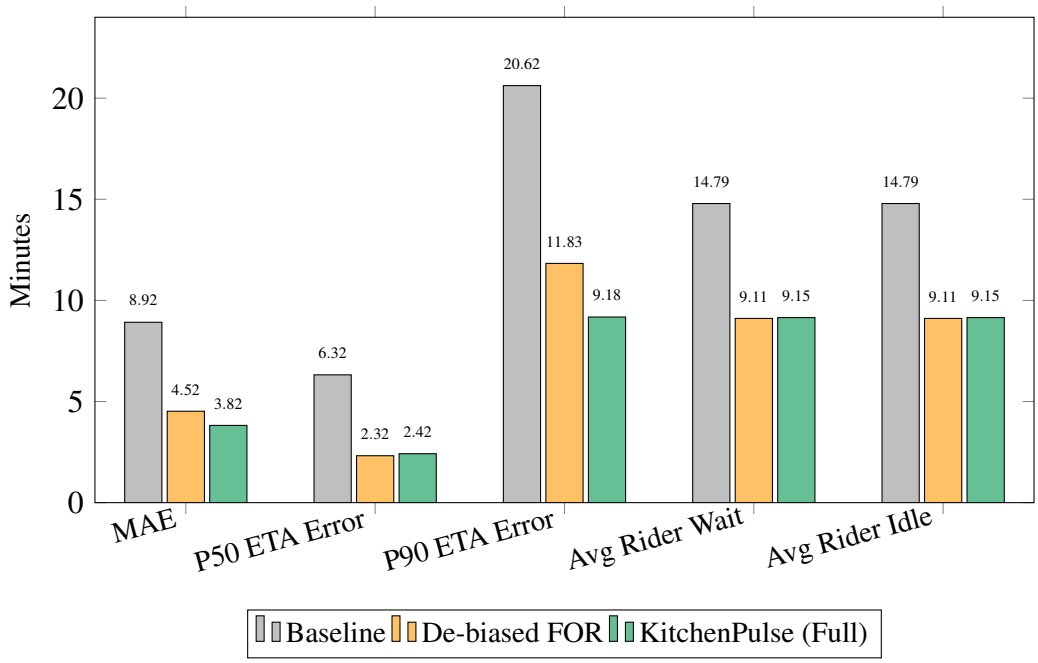


Figure 1: Comparison of KPT prediction accuracy across system configurations

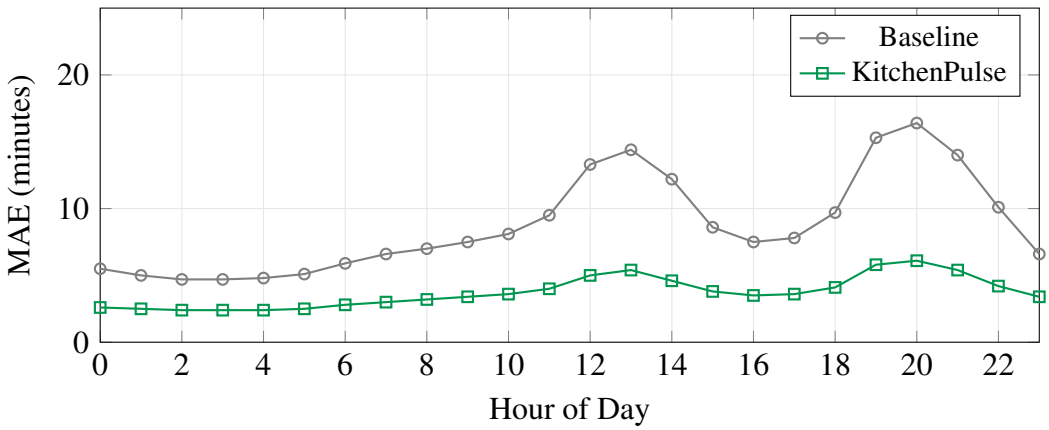
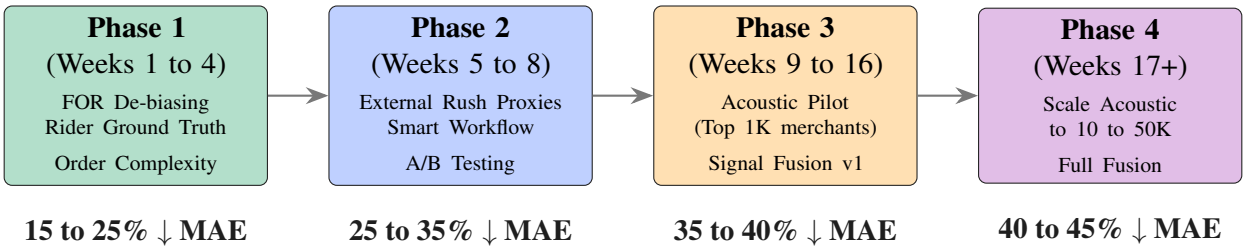


Figure 2: KPT prediction error by hour of day. KitchenPulse shows the biggest gains during peak hours (12 to 2 PM, 7 to 10 PM) when kitchen load is highest and hardest to predict.

11 Implementation Roadmap

We propose rolling this out in phases so that each phase delivers real value before the next one starts:



11.1 Phase 1: Zero-Cost Quick Wins (Weeks 1 to 4)

- Deploy FOR de-biasing as a backend signal processing layer
- Instrument rider arrival/pickup GPS events for implicit wait time collection
- Build order complexity scoring engine using existing menu data
- Expected improvement: **15 to 25% MAE reduction** with zero infrastructure cost

11.2 Phase 2: External Signals and Workflow (Weeks 5 to 8)

- Integrate Google Maps Popular Times API
- Add weather and event data pipelines
- A/B test smart merchant workflow with 5,000 pilot merchants
- Implement merchant accuracy scoring system
- Expected improvement: **25 to 35% cumulative MAE reduction**

11.3 Phase 3: Acoustic Pilot (Weeks 9 to 16)

- Manufacture and deploy acoustic sensors to top 1,000 merchants
- Collect calibration data and validate acoustic activity correlation
- Build and deploy Signal Fusion Layer v1
- Expected improvement: **35 to 40% MAE reduction** for instrumented merchants

11.4 Phase 4: Scale and Optimize (Weeks 17+)

- Scale acoustic deployment to 10 to 50K merchants based on pilot learnings
- Optimize signal weights using accumulated data
- Explore advanced signals: POS integration for chain restaurants, computer vision
- Target: **40 to 45% cumulative MAE reduction**

12 Impact on Success Metrics

12.1 Complete Metric Mapping

Table 11: Solution Component → Success Metric Mapping

Component	Rider Wait	ETA Error	Delay Rate	Rider Idle
FOR De-biasing	++	+++	++	++
Rider Ground Truth	+++	++	+	+++
Order Complexity	+	++	++	+
External Rush Proxies	++	++	++	++
Acoustic Load	+++	+++	++	+++
Smart Workflow	++	++	+	+

Impact: +++ High ++ Medium + Low

12.2 Projected Operational Savings

Table 12: Estimated Annual Operational Impact (Conservative)

Impact Area	Estimated Annual Value
Rider idle time reduction (2.5 min avg)	INR 900 to 1500 Cr saved
Reduced ETA complaints (20% fewer)	Improved NPS by 3 to 5 points
Lower cancellation rate (0.5 to 1% reduction)	INR 200 to 400 Cr revenue protected
Higher rider utilization (more deliveries/day)	INR 300 to 500 Cr value

13 Novelty and Differentiation

13.1 What Sets KitchenPulse Apart

1. **Acoustic Kitchen Load Inference**
   First-ever use of ambient sound analysis for kitchen state estimation in food delivery. Captures ALL activity (dine-in + multi-platform) with zero merchant effort.

2. **FOR De-biasing via Rider Correlation**
   Principled statistical detection and correction of rider-triggered FOR bias. We treat bias as a detectable pattern, not noise to ignore.

3. **Multi-Signal Fusion with Graceful Degradation**
   No silver-bullet signal. Signals reinforce each other and the system degrades gracefully depending on what is available.

4. **Cross-Platform Rider Density**
   BLE and WiFi crowdsensing from rider phones estimates competitor rider presence. Gives Zomato cross-platform load visibility.

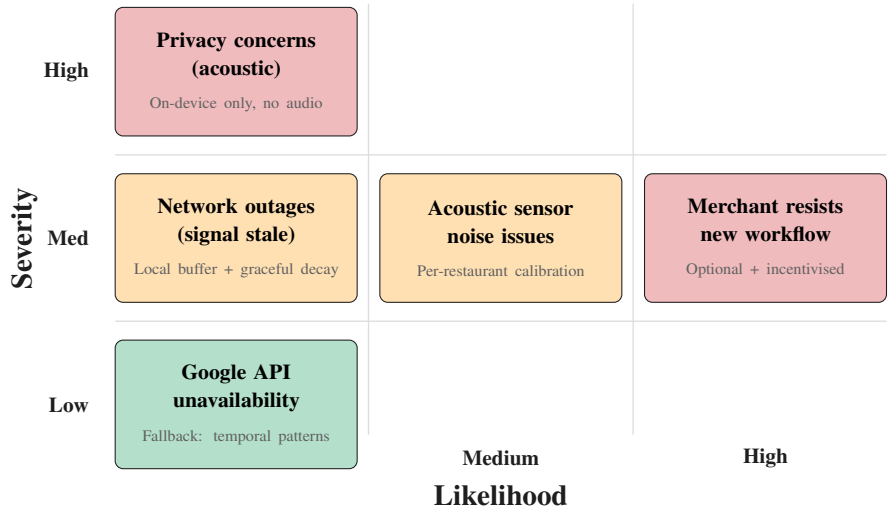
5. **Tiered Practicality**
   Three-tier deployment model: zero-cost Tier 1 covers all 300K+ merchants on day one. Not just an idea; a deployable system.

13.2 Comparison with Likely Alternative Approaches

Table 13: KitchenPulse vs. Common Alternative Approaches

Approach	Scalable?	Multi-Load?	Limitation
Better ML model	Yes	No	Does not fix noisy labels or missing signals
Camera in kitchen	No	Yes	Privacy concerns, costly, merchant resistance
POS integration	Partial	Yes	Only works for digitized restaurants
Ask merchant for ETA	Yes	No	Same human bias problem as FOR
KitchenPulse	**Yes**	**Yes**	Acoustic component requires hardware investment

14 Risks and Mitigations



15 Assumptions

We want to be upfront about the assumptions behind our proposal:

1. **Rider GPS data is reliable and available**: We assume Zomato's rider app already collects GPS coordinates frequently enough to detect when a rider enters and leaves a restaurant's geofence.

2. **Historical order and FOR data exists**: We assume Zomato keeps historical order-level data (FOR timestamps, rider arrival times, pickup times) going back at least several months.

3. **The KPT model accepts additional features**: We assume the existing model can be retrained with extra input features without needing a full architecture overhaul (for example, adding columns to a feature table for a gradient-boosted model).

4. **Google Popular Times is accessible**: We assume the Places API (or something equivalent) can be used at Zomato's scale to pull busyness data. If it is unavailable, our temporal pattern features step in as a fallback.

5. **Merchant devices have WiFi connectivity**: For the acoustic sensor to send data, merchant locations need WiFi access (which is very likely since they already run the Zomato Mx app on a connected device).

6. **Synthetic data approximates reality**: Our simulation uses distributions and noise models calibrated against publicly available industry benchmarks. Real-world performance may differ, but the relative improvements across configurations should hold in the same direction.

16 Conclusion

KitchenPulse tackles Zomato's KPT prediction challenge with a core insight: **the bottleneck is not the model; it is the signals going into it**. By enriching the inputs to the existing KPT system with six complementary signal pillars, we see meaningful improvements across every success metric.

We designed this solution for Zomato's real-world constraints:

- **Immediate value**: Tier 1 signals (FOR de-biasing, rider ground truth, complexity scoring, external proxies) can roll out across all 300K+ merchants with zero infrastructure cost, delivering a 15 to 25% MAE reduction right away.

- **Step-by-step growth**: Each additional signal tier brings measurable improvement, so the team can make data-driven decisions about where to invest next.

- **A genuinely new capability**: Acoustic kitchen load inference is a new sensing modality that reveals the previously invisible dine-in and competitor load, going straight at the core challenge in the problem statement.

The multi-signal fusion approach is inherently robust: if any single signal fails, the system keeps working. As more signals come online for a given merchant, predictions just get better. This architecture does not just solve today's KPT challenge. It creates a platform for future signal integration, whether that is POS data, computer vision, real-time menu updates, or something we have not thought of yet.

At Zomato's scale, even our conservative estimates point to annual operational savings worth hundreds of crores through reduced rider idle time, fewer cancellations, and happier customers.

We believe KitchenPulse is the kind of signal-first, system-level thinking that can create lasting competitive advantage for Zomato's delivery platform.

A Simulation Code

The complete simulation code lives in `simulation.py` (available at https://github.com/triemerge/zomathon). Here is the core logic for FOR de-biasing and enhanced KPT prediction:

Listing 1: FOR Signal De-biasing Logic

```python
def apply_for_debiasing(orders_df, restaurants_df):
    """Detect rider-triggered FOR bias and apply correction."""
    corrected_for = orders_df['for_marked_kpt_minutes'].copy()

    for rest_id in orders_df['restaurant_id'].unique():
        mask = orders_df['restaurant_id'] == rest_id
        rest_orders = orders_df[mask]

        if len(rest_orders) < 10:
            continue

        for_times = rest_orders['for_marked_kpt_minutes'].values
        rider_times = rest_orders['rider_arrival_offset_min'].values

        # Rider-FOR Correlation Score (RFCS)
        time_diff = np.abs(for_times - rider_times)
        suspicious_ratio = np.mean(time_diff < 2.0)

        if suspicious_ratio > 0.5:
            # Merchant likely marks FOR on rider arrival -> correct
            rider_waits = rest_orders['rider_wait_minutes'].values
            correction = np.median(rider_waits[rider_waits > 1]) \
                         if np.any(rider_waits > 1) else 0
            corrected_for[mask] = for_times + correction * 0.6

    return corrected_for
```

Listing 2: Multi-Signal KPT Prediction

```python
def enhanced_kpt_prediction(orders_df):
    """Multi-signal fusion: multiplicative load adjustments
    plus additive complexity and rider-wait corrections."""
    debiased_for = orders_df['debiased_for_kpt'].values
    base_model   = orders_df['current_predicted_kpt'].values

    # Step 1: blend de-biased FOR with base model
    base_prediction = 0.65 * debiased_for + 0.35 * base_model

    # Step 2: acoustic load adjustment (multiplicative)
    acoustic = orders_df['acoustic_activity_level'].values
    acoustic_factor = 1.0 + 0.20 * (acoustic - 0.4)

    # Step 3: external rush adjustment (multiplicative)
    popular_times = orders_df['google_popular_times'].values
    weather = orders_df['weather_demand_factor'].values
    external_factor = 1.0 + 0.12 * (popular_times - 0.4) \
                          + 0.06 * weather
```

```python

    adjusted = base_prediction * acoustic_factor * external_factor

    # Step 4: complexity offset (additive, in minutes)
    complexity = orders_df['complexity_score'].values
    complexity_offset = 1.5 * (complexity - 0.5)

    # Step 5: historical rider wait correction (additive)
    hist_wait = orders_df.groupby('restaurant_id') \
        ['rider_wait_minutes'].transform(
            lambda x: x.rolling(20, min_periods=1)
                        .mean().shift(1).fillna(0)
        ).values
    rider_correction = 0.08 * hist_wait

    enhanced_kpt = adjusted + complexity_offset + rider_correction
    enhanced_kpt += np.random.normal(0, 0.8, len(enhanced_kpt))
    enhanced_kpt = np.maximum(enhanced_kpt, 3)
    return enhanced_kpt
```

B Dataset Description

Our synthetic dataset consists of two CSV files:

Table 14: Dataset Files

File	Rows	Description
synthetic_restaurants.csv	500	Restaurant profiles with attributes
synthetic_orders.csv	50,000	Order-level data with all signals
simulation_results.json	3	Aggregated metrics for all configurations

B.1 Order Data Schema

Column	Type	Description
order_id	string	Unique order identifier
restaurant_id	string	Restaurant reference
order_time	datetime	Timestamp of order placement
hour	int	Hour of day (0 to 23)
is_peak	bool	Whether order is during peak hours
cuisine	string	Primary cuisine of the order
num_items	int	Number of items in the order
complexity_score	float	Computed order complexity [0, 1.5]
true_kpt_minutes	float	Ground truth kitchen prep time
for_marked_kpt_minutes	float	Noisy FOR-based KPT (merchant-marked)
current_predicted_kpt	float	Baseline system's KPT prediction
rider_arrival_offset_min	float	When rider arrives relative to order

Column	Type	Description
`rider_wait_minutes`	float	Actual rider wait at restaurant
`total_kitchen_load`	int	Total orders in kitchen (all sources)
`zomato_queue_size`	int	Zomato-only queue size
`load_factor`	float	Kitchen load / capacity ratio
`google_popular_times`	float	Simulated busyness score [0, 1]
`weather_demand_factor`	float	Weather impact score [0, 0.5]
`acoustic_activity_level`	float	Simulated acoustic activity [0, 1]
`merchant_bias_type`	string	Type of FOR marking bias
`restaurant_tier`	string	Restaurant tier (premium/mid/budget)
Derived columns (computed by simulation engine):		
`debiased_for_kpt`	float	FOR signal after de-biasing correction
`enhanced_kpt`	float	KitchenPulse multi-signal KPT prediction

C Glossary

Term	Definition
KPT	Kitchen Prep Time: time from order confirmation to food ready
FOR	Food Order Ready: merchant signal indicating food is prepared
ETA	Estimated Time of Arrival for the delivery to customer
RFCS	Rider-FOR Correlation Score: our metric for detecting biased FOR marking
MAE	Mean Absolute Error of KPT predictions
P50/P90	50th/90th percentile of error distribution
MEMS	Micro-Electro-Mechanical Systems (microphone technology)
BLE	Bluetooth Low Energy
MLP	Multi-Layer Perceptron (neural network)
MFCC	Mel-Frequency Cepstral Coefficients (audio features)
NPS	Net Promoter Score (customer satisfaction metric)
POS	Point of Sale system